

Projet Cassiopée *DiveMonitor* - Documentation

Antonin Fruchet, Loup Benois

Février à Mai 2026

Dépôt Git : [Lien](#)

Table des matières

1	Introduction	1
2	Installations et prérequis	2
2.1	Qt & Qt Creator	2
2.1.1	Création d'un compte Qt	2
2.1.2	Installation de Qt et Qt Creator	2
2.2	Outils de développements Android	4
2.2.1	JDK, Android NDK et Android SDK	4
2.2.2	Configuration de l'appareil Android	5
2.3	Installations des Dépendances	6
2.3.1	Installation de libdivecomputer	6
3	Fonctionnement technique de l'application	7
3.1	Arborescence	7
3.2	Vue d'ensemble de l'architecture	7
3.3	Interface graphique QML	7
3.4	Communication entre QML et C++	7
3.5	Gestion des ordinateurs de plongée	8
3.6	Importation des plongées avec libdivecomputer	8
3.7	Stockage local des données	8
3.8	Affichage des plongées et des graphiques	9
3.9	Spécificités Android et JNI	13
4	Annexes	13
4.1	Annexe 1 - Script de compilation de <i>libdivecomputer</i> pour Android	13

1 Introduction

DiveMonitor est une application mobile de gestion de plongée développée en C++ et Qt Quick, s'inspirant de Subsurface. Elle permet l'importation directe de données depuis un ordinateur de plongée vers une interface fluide. L'application centralise l'historique des immersions et propose, pour chaque session, un graphique interactif permettant une analyse précise des profils de profondeur, de vitesse et de température.

2 Installations et prérequis

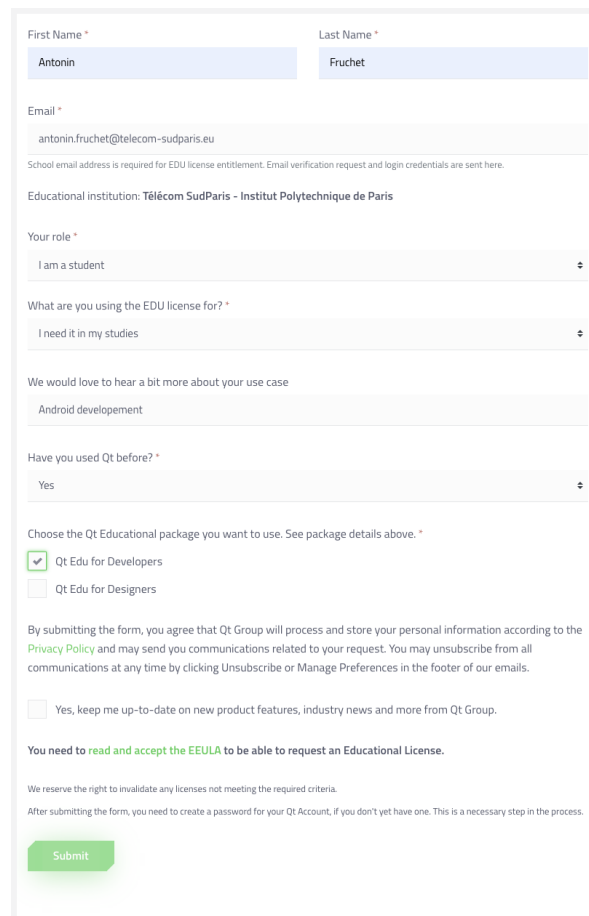
Cette section est détaillée les étapes d'installations de tout ce qui est nécessaire au fonctionnement et à la compilation du projet.

2.1 Qt & Qt Creator

Si vous disposez déjà d'une licence Qt Entreprise ou Education, passez à la sous-section 2.1.2.

2.1.1 Création d'un compte Qt

1. Se rendre sur [cette page](#).
2. Renseigner les informations demandées comme suit :



The image shows a web form for requesting a Qt Educational License. The form is titled 'First Name *' and 'Last Name *' at the top. Below these are input fields for 'Antonin' and 'Fruchet'. The 'Email *' field contains 'antonin.fruchet@telecom-sudparis.eu'. A note states: 'School email address is required for EDU license entitlement. Email verification request and login credentials are sent here.' The 'Educational institution' field is filled with 'Télécom SudParis - Institut Polytechnique de Paris'. The 'Your role *' dropdown menu is set to 'I am a student'. The 'What are you using the EDU license for? *' dropdown menu is set to 'I need it in my studies'. A text box for 'We would love to hear a bit more about your use case' contains 'Android development'. The 'Have you used Qt before? *' dropdown menu is set to 'Yes'. Under 'Choose the Qt Educational package you want to use. See package details above. *', the 'Qt Edu for Developers' option is selected with a green checkmark, while 'Qt Edu for Designers' is unselected. A paragraph of terms and conditions follows, mentioning the Privacy Policy and the right to unsubscribe. A checkbox for 'Yes, keep me up-to-date on new product features, industry news and more from Qt Group.' is present. A bold statement reads: 'You need to read and accept the EEULA to be able to request an Educational License.' At the bottom, a small note says: 'We reserve the right to invalidate any licenses not meeting the required criteria. After submitting the form, you need to create a password for your Qt Account. If you don't yet have one. This is a necessary step in the process.' A green 'Submit' button is at the bottom.

FIGURE 1 – Formulaire pour l'accès à Qt EDU

3. Finaliser l'inscription, et valider la création du compte avec le mail de confirmation (peut prendre un petit peu de temps à arriver).

2.1.2 Installation de Qt et Qt Creator

1. Se rendre sur sa page de profil et aller dans l'onglet **Qt Download** puis télécharger **Qt Online Installer**.

2. Ceci fait, lancer le programme, se connecter et avancer jusqu'à cette page :

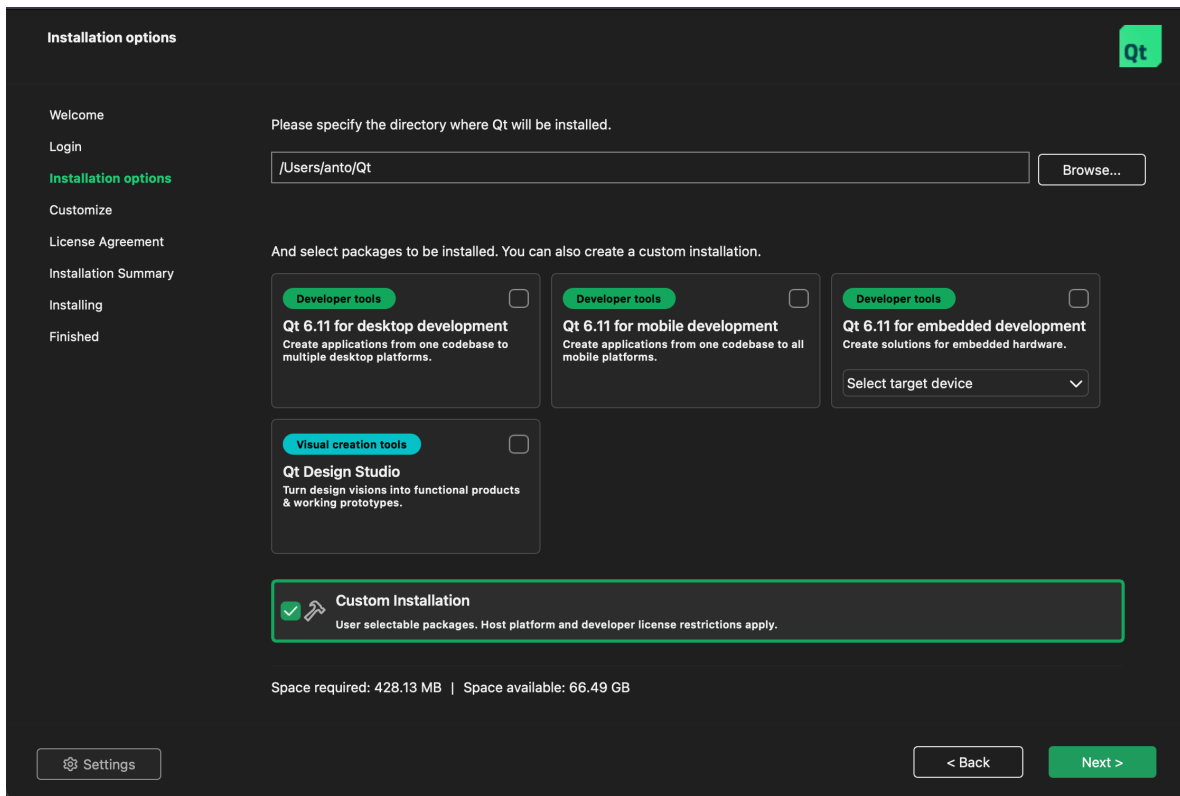


FIGURE 2 – Choix du type d'installation

3. Bien sélectionner **Custom Installation**.
4. Sur la page suivante cocher au moins ces cases :

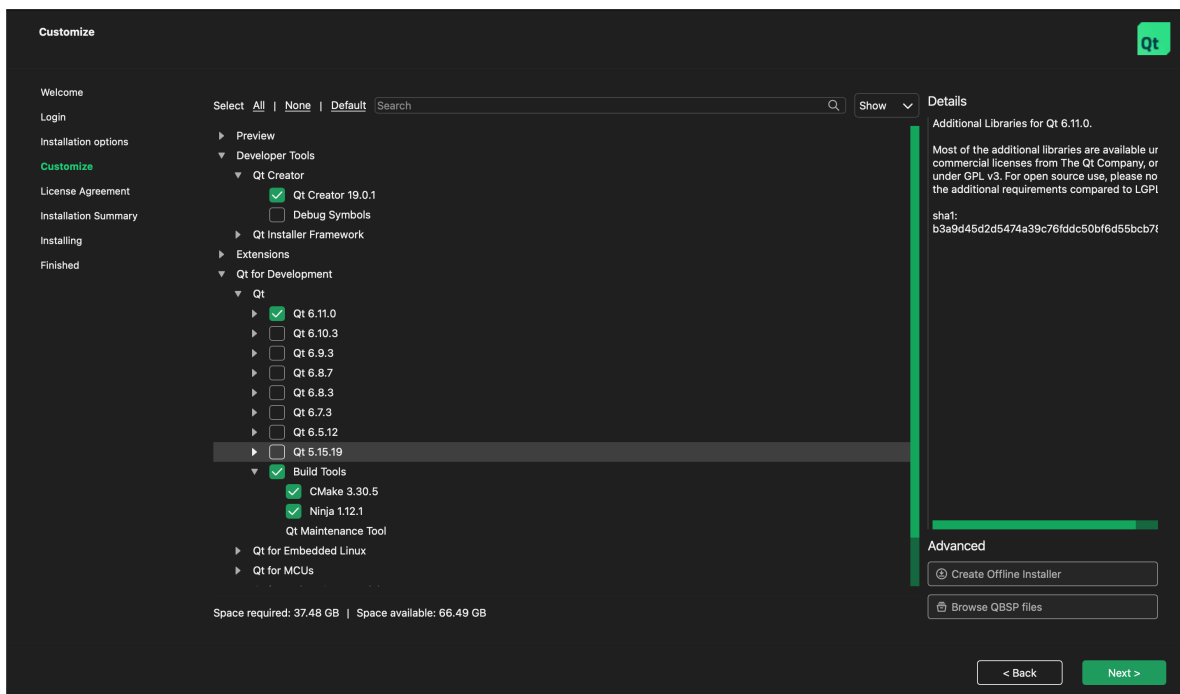


FIGURE 3 – Choix des éléments Qt à installer

5. Choisir au moins un **Qt 6.10.3**.
6. Continuer et terminer l'installation.

Qt et Qt Creator sont maintenant installés correctement sur votre machine.

Essayer tout de même de compiler et d'exécuter un des projets exemples fournis par Qt, tout devrait en théorie bien se passer.

Note : À tout moment il est possible d'installer/désinstaller de nouveaux modules ou application Qt via *Qt Maintenance Tool* qui s'est installé en même temps que le reste. L'interface est très semblable à l'installer.

2.2 Outils de développements Android

Cette étape est obligatoire pour compiler et déployer l'application sur un appareil Android.

2.2.1 JDK, Android NDK et Android SDK

Pour configurer l'environnement Android, aller dans **Preferences > SDKs** puis dans l'onglet **Android**. Cette page doit s'ouvrir, suivez les étapes suivantes :

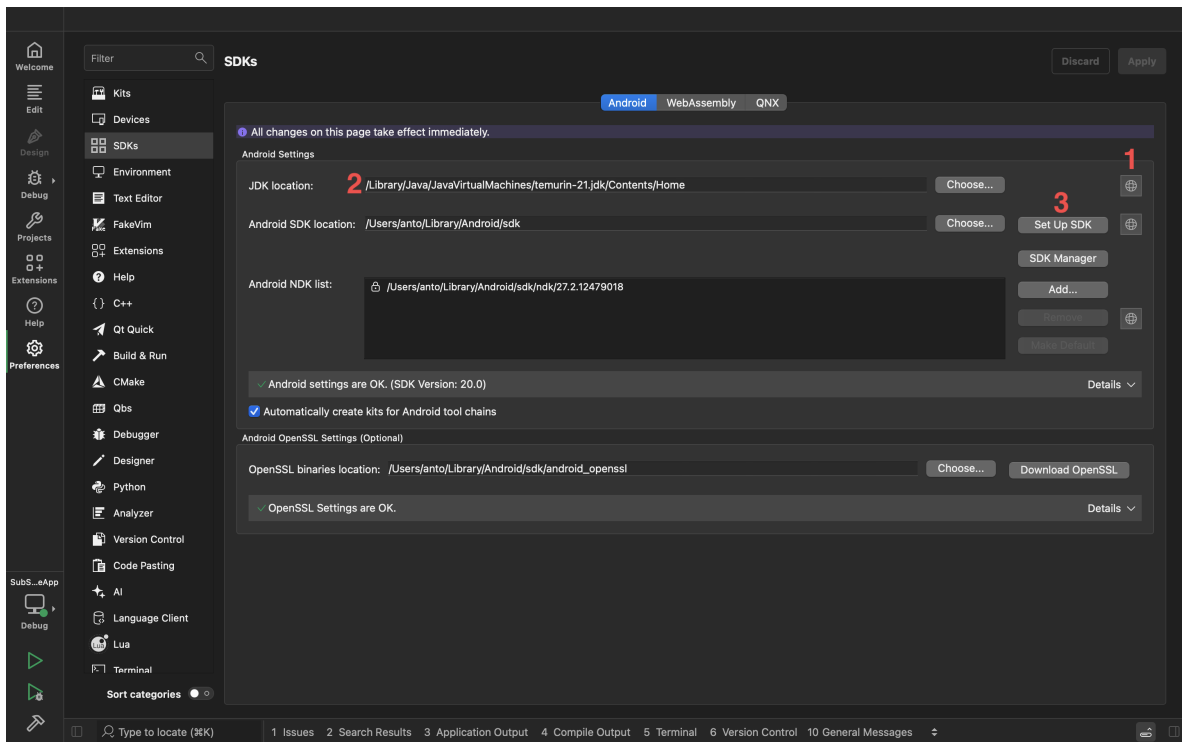


FIGURE 4 – Panneau de configuration de l’environnement Android

1. Installer le *Java Development Kit (JDK)* en cliquant sur le bouton **1**. Celui-ci renvoie vers une page de téléchargement ou la dernière version peut être téléchargée. Télécharger et installer le JDK.
2. Normalement, une fois le JDK installé le champ **2** devrait s’auto remplir, **Si ce n’est pas le cas**, renseigner le chemin manuellement.
3. Le *Software Development Kit (SDK)* a lui été installé en même temps que Qt et doit déjà apparaître. Si c’est effectivement le cas, cliquer sur le bouton **3** pour configurer les outils du SDK. Cela devrait aussi se charger des choses manquantes pour OpenSSL.

Après cela QtCreator devrait afficher que tout est bon.

2.2.2 Configuration de l’appareil Android

Il est possible, avec Qt Creator, de déployer l’application directement sur un appareil Android physique en le branchant à votre machine.

Pour ce faire, l’appareil Android doit être mis en mode développeur. *(NB : Ceci n’est pas une manipulation dangereuse comme acquérir les droits root, le mode développeur est activable ou désactivable à tout moment.)*

1. Aller dans **Paramètres > À propos du téléphone** (ou équivalent)
2. Trouver **Numéro de version** (Build Number) puis cliquer 7 fois dessus de suite.
3. Normalement le téléphone est maintenant en mode développeur, et un nouveau menu **Outils de développement** doit être disponible.

4. Dans ce nouveau menu, cocher **Débogage USB** pour permettre la connexion via USB à votre machine.

Brancher maintenant l'appareil à votre machine et vous devriez le voir apparaître dans QtCreator sous l'onglet **Devices** des Préférences.

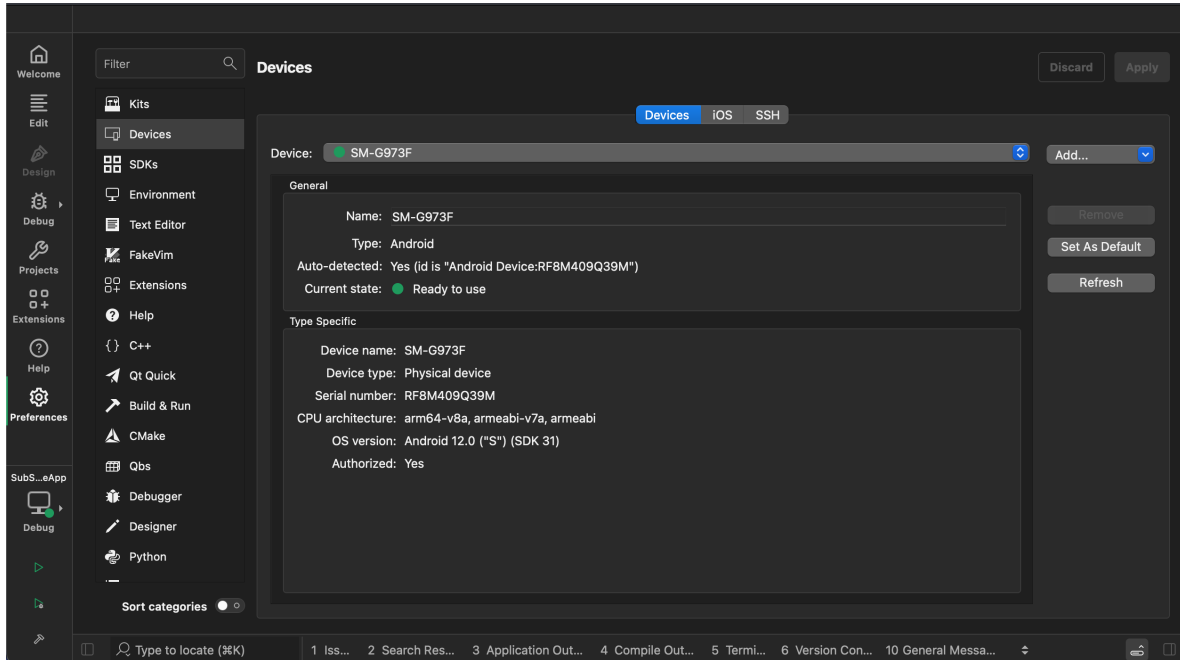


FIGURE 5 – Appareil branché et actif dans Qt Creator

Il est maintenant possible de compiler pour Android et déployer une application directement sur l'appareil Android depuis votre machine quelle qu'elle soit.

2.3 Installations des Dépendances

Le projet utilise donc Qt et ses modules Quick, QuickControls2, Graphs, Sql et Concurrent (installés avec Qt normalement, si ce n'est pas le cas retourner l'installer depuis Qt Maintenance Tool), ainsi que la librairie `libdivecomputer` (version 0.9.0).

2.3.1 Installation de `libdivecomputer`

Libdivecomputer est disponible pour Linux et Mac-OS sur [Homebrew](#) uniquement. L'installer de cette manière est pratique pour le développement sur votre machine, en revanche cela ne permettra pas de compiler la librairie pour Android. (Il est possible de compiler la librairie à la main aussi à partir des binaires disponibles [ici](#))

La librairie est donc précompilée pour Android et incluse dans son entièreté dans la hiérarchie du projet sous `lib/android_libs`.

Si votre appareil n'est pas le même type d'android que ce pour quoi la librairie a été compilée, il est possible de la recompiler grâce au script de compilation de l'Annexe 1. (Ce script est présent sous *lib/android_libs/libdivecomputer-0.9.0* et s'appelle *build_android.sh*)

3 Fonctionnement technique de l'application

3.1 Arborescence

Le projet est organisé comme suit sous *project/*

```
DiveMonitorApp/
├── src/ .....Code source C++
├── qml/ .....Code source QML (interface)
│   ├── pages/
│   ├── overlay/
│   └── components/
├── resources/ .....logos, icones etc
├── lib/ .....Dépendances (libdivecomputer pour android)
└── android/ .....Configuration pour le build et la compatibilité android
```

3.2 Vue d'ensemble de l'architecture

L'application est organisée autour de trois grandes couches. La première est l'interface graphique, écrite en QML, qui gère l'affichage des pages, les interactions utilisateur et la navigation. La deuxième est la couche C++, qui contient la logique métier de l'application : connexion à un ordinateur de plongée, importation des données, modèles exposés à QML et accès à la base de données. Enfin, la troisième couche correspond aux dépendances natives, principalement *libdivecomputer*, utilisée pour communiquer avec les ordinateurs de plongée.

Cette séparation permet de garder une interface relativement simple côté QML, tandis que les opérations plus complexes, comme l'importation ou le parsing des plongées, sont centralisées dans le code C++.

3.3 Interface graphique QML

L'interface est définie dans le dossier *qml/*. Le fichier principal est *Main.qml*. Il contient la fenêtre principale de l'application, le **StackView** utilisé pour naviguer entre les différentes pages, ainsi que les objets partagés comme les couleurs, les préférences utilisateur et l'état de l'ordinateur de plongée sélectionné.

Les pages principales sont placées dans *qml/pages/*. On y trouve par exemple la liste des plongées, la page de détails, la page graphique et la page d'importation. Les composants réutilisables sont séparés dans *qml/components/* et les éléments affichés par-dessus les pages, comme le header ou le bouton d'ajout, sont dans *qml/overlay/*.

3.4 Communication entre QML et C++

La communication entre l'interface QML et la logique C++ repose sur le système de propriétés et de méthodes invocables de Qt (mot clé **Q_INVOKABLE** dans le C++). Dans *main.cpp*, l'objet **DiveComputerWrapper** est exposé au moteur QML sous le nom **dcWrapper**. Cela permet aux fichiers QML d'appeler directement certaines méthodes C++ comme **connectToDevice()**, **disconnectDevice()**, **refreshPorts()** ou **importDivesAsync()**.

Les modèles de données, comme `DiveListModel` et `SampleModel`, sont également enregistrés auprès du moteur QML. Ils permettent d’afficher les plongées et les échantillons sous une forme exploitable directement par les vues QML.

3.5 Gestion des ordinateurs de plongée

La classe centrale de cette partie est `DiveComputerWrapper`. Elle joue le rôle d’intermédiaire entre l’interface et *libdivecomputer*. Elle maintient l’état de connexion, la liste des appareils supportés, les ports disponibles et les objets internes nécessaires à la communication avec un ordinateur de plongée.

Lorsqu’un utilisateur choisit une marque, un modèle et un mode de connexion, l’application recherche le descripteur correspondant dans *libdivecomputer*. Ce descripteur permet ensuite d’ouvrir une communication avec l’appareil. Sur desktop, l’ouverture se fait principalement via un port série classique. Sur Android, une couche spécifique est utilisée pour gérer les périphériques USB. (voir section 3.9 pour plus de détails sur la connexion USB avec Android).

3.6 Importation des plongées avec *libdivecomputer*

L’importation des plongées s’appuie sur *libdivecomputer*. Une fois l’appareil connecté, l’application ouvre une session de communication avec l’ordinateur de plongée. Les plongées sont ensuite récupérées puis analysées afin d’en extraire les informations principales : date, durée, profondeur, température et échantillons du profil de plongée.

L’importation est lancée de manière asynchrone avec *QtConcurrent*. Cela évite de bloquer l’interface graphique pendant la récupération des données, qui peut prendre plusieurs secondes selon l’appareil et le nombre de plongées.

L’extraction des plongées s’opère avec la fonction `dc_device_foreach` de *libdivecomputer*. Cette fonction itère sur chaque plongée trouvée dans l’appareil de plongée et exécute une fonction callback (passée en second argument) qui nous permet de traiter à notre guise les données de la plongée. En l’occurrence on enregistre les données dans une structure `DiveData` (voir header `DiveDataStructure.h`). A chaque itération il est nécessaire d’appeler la fonction `dc_parser_sample_foreach` qui extrait désormais l’ensemble des points (temps, profondeur, température) depuis la plongée en cours. Cette fonction marche de la même manière que `dc_device_foreach` avec une fonction callback.

La fonction `dc_device_foreach` arrête d’itérer quand son callback renvoie 0 ou quand toutes les plongées ont été parcourues. Ainsi on gère au début du callback le fingerprint (sorte d’id unique) de plongée, si ce callback ne correspond pas à celui déjà enregistré, on l’enregistre car il s’agit d’une potentielle plongée la plus récente et on continue à itérer. A la fin on récupère donc le fingerprint le plus récent qu’on insère en BDD à la place de l’ancien. Les fingerprints sont propres à un appareil, et donc stockés comme tel dans la base.

Les méthodes et fonctions d’import des plongées sont situées dans *DiveComputerWrapper.c* :

- `void DiveComputerWrapper::importDives()` : méthode notée `Q_INVOKABLE`, point d’entrée pour le téléchargement des plongées depuis l’interface.
- `int dive_callback(...)` : callback pour les plongées (pour `dc_device_foreach`)
- `void dive_sample_callback(...)` : callback pour les échantillons temporels (pour `dc_parser_sample_foreach`)

3.7 Stockage local des données

Les plongées importées sont stockées localement grâce au module SQL de Qt et sont donc persistentes d’une session à une autre. La classe `DiveDatabase` centralise l’accès à la base de données. Elle permet d’insérer de nouvelles plongées, de récupérer la liste des plongées existantes et de stocker certaines informations nécessaires aux imports suivants, comme le *fingerprint* associé à un ordinateur de plongée.

Voir directement le header de la classe `DiveDatabase` pour voir les actions possibles sur la base de données.

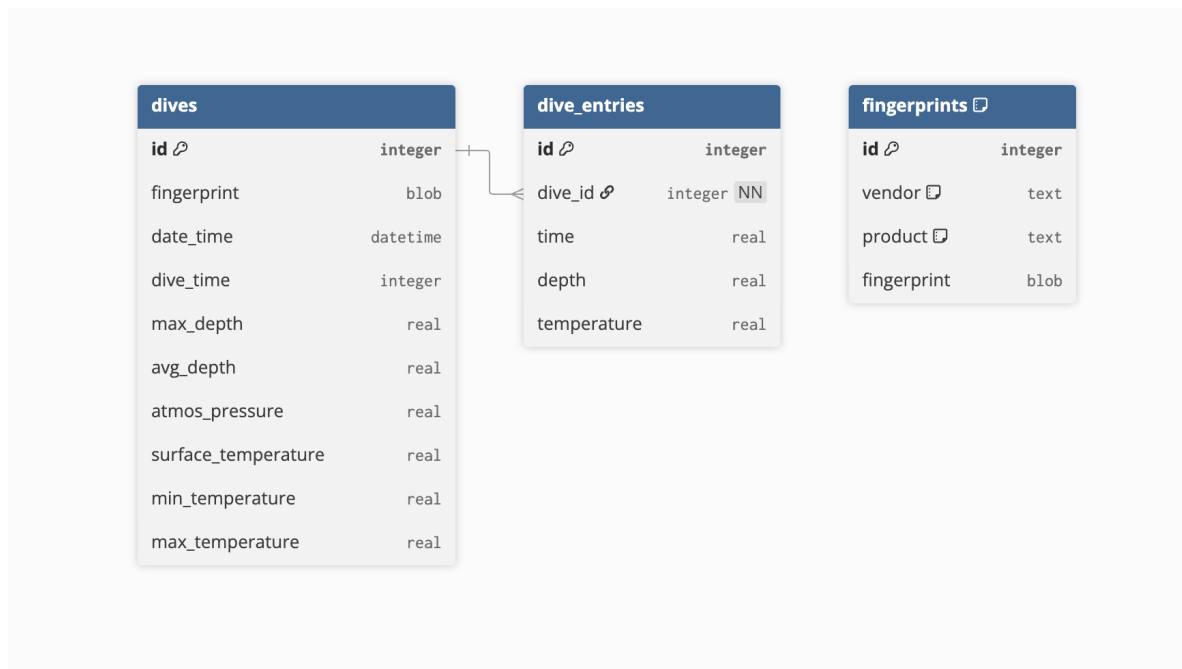


FIGURE 6 – Architecture de la base de données

La fichier de base de donnée est créé dans le Local Storage de l'application :

- **Android** : `/data/user/0/org.subsurfadedivelog.mobile/files/`
- **Linux** : `~/.local/share/appDiveMonitorApp/`
- **MacOS** : `~/Library/ApplicationSupport/appDiveMonitorApp/`
- **Windows** : `C:\Users\Utilisateur\AppData\Local\NomDeLApplication`

3.8 Affichage des plongées et des graphiques

L'affichage d'une plongée se fait en deux temps. L'utilisateur arrive d'abord sur la liste des plongées enregistrées, construite à partir du modèle `DiveListModel`. Lorsqu'il clique sur une plongée, l'objet correspondant est transmis à la page `DiveDetails.qml`. Cette page affiche à la fois les informations générales de la plongée et un graphique permettant d'analyser son profil temporel.

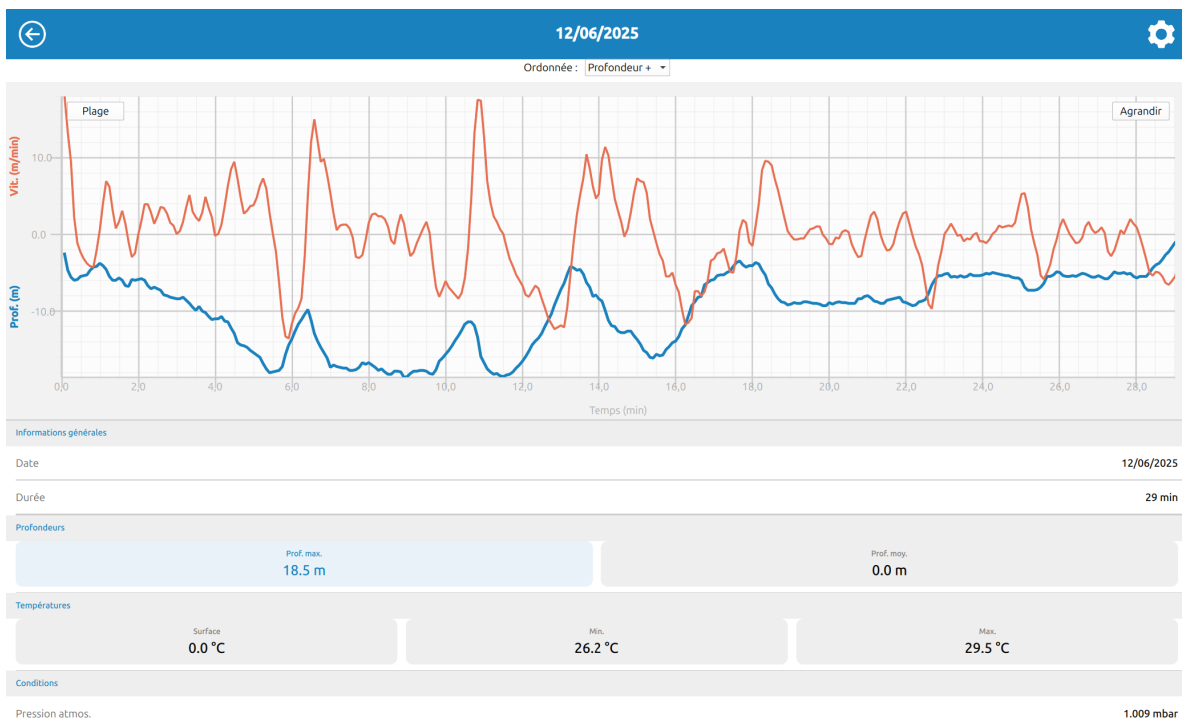


FIGURE 7 – Page de détail d’une plongée avec le graphe et les informations générales

Le graphique est encapsulé dans un composant QML réutilisable : *DiveGraphView.qml*. Ce composant est utilisé une première fois dans *DiveDetails.qml*, où il occupe la partie supérieure de la page de détail, puis une seconde fois dans *DiveGraphPage.qml*, qui sert à afficher le graphe en plus grand. Le bouton **Agrandir** de la page de détail ouvre cette seconde page en lui transmettant la plongée courante ainsi que le mode d’affichage actuellement sélectionné. Cela permet de conserver le même graphe et les mêmes données, tout en offrant plus de place à l’utilisateur pour naviguer dans le profil de plongée.

Le tracé du graphe repose sur le module *Qt Graphs*. Côté QML, le composant principal est un **GraphsView**. Les courbes affichées dans ce **GraphsView** ne sont pas créées directement depuis le QML, mais depuis un modèle C++ appelé **SampleModel**. Cette classe hérite de **QLineSeries**, ce qui lui permet d’être utilisée directement comme une série de points dans le graphe QML. Elle est enregistrée dans *main.cpp* avec **qmlRegisterType**, puis instanciée dans *DiveGraphView.qml* comme un composant QML classique.

Au chargement du composant, **SampleModel** reçoit l’identifiant de la plongée avec la méthode **setEntries(root.dive.id)**. Cette méthode va chercher les échantillons temporels de la plongée dans la base de données, via **DiveDatabase::getDiveEntries**. Les points sont ensuite conservés en mémoire dans **m_rawEntries**. L’intérêt de ce fonctionnement est que les données brutes ne sont chargées qu’une seule fois depuis la base. Lorsqu’on change le mode d’affichage du graphe, le modèle recalcule simplement les points de la courbe sans refaire de requête SQL.

Chaque point du graphe correspond à une entrée de type **DiveEntry**. Son abscisse est toujours le temps, exprimé en secondes depuis le début de la plongée. Son ordonnée dépend du mode sélectionné :

- **Profondeur** : l'ordonnée correspond à la profondeur en mètres. La valeur est inversée dans le modèle C++ afin que la surface apparaisse en haut du graphe et que la descente soit visuellement orientée vers le bas. Les labels affichés sur l'axe utilisent ensuite la valeur absolue, pour que l'utilisateur lise bien une profondeur positive.
- **Température** : l'ordonnée correspond directement à la température enregistrée pour l'échantillon courant.
- **Vitesse** : l'ordonnée correspond à une vitesse verticale en mètres par minute. Elle est calculée dans `SampleModel` à partir de l'évolution de la profondeur autour du point courant.
- **Profondeur + vitesse** : deux séries sont affichées en même temps. La première représente la profondeur, et la seconde la vitesse.

Le calcul de la vitesse est volontairement fait côté C++ plutôt que côté QML, car il dépend des données brutes de la plongée. Pour un point donné, le modèle récupère plusieurs échantillons voisins autour du point courant, puis calcule une pente locale entre le temps et la profondeur. Cette pente est ensuite convertie en mètres par minute. Cette méthode permet d'obtenir une vitesse plus stable qu'une simple différence entre deux points consécutifs. Si le nombre de points autour de l'échantillon est insuffisant, le code utilise alors le point précédent ou le point suivant comme solution de secours.

Le changement de mode d'affichage est contrôlé par une `ComboBox` présente au-dessus du graphe en affichage normal. En mode plein écran paysage, ce sélecteur est placé dans l'overlay du graphe afin de garder le plus de place possible pour la courbe. Les modes disponibles sont définis directement dans *DiveGraphView.qml* : *Profondeur + vitesse*, *Profondeur*, *Vitesse* et *Température*. Quand l'utilisateur choisit un mode, la fonction QML `applyDisplayMode()` est appelée, puis le `GraphsView` met à jour le mode courant, demande au `SampleModel` de recalculer la série et réinitialise les bornes visibles des axes.

Les axes sont gérés avec deux `ValueAxis`. L'axe horizontal représente le temps, de 0 à la durée totale de la plongée. L'axe vertical dépend du mode d'affichage courant et utilise les bornes calculées par `SampleModel` avec les propriétés `displayMin` et `displayMax`. Les labels des axes sont redessinés manuellement dans l'overlay. Cela permet d'adapter plus finement leur format : le temps est affiché en minutes, la profondeur est affichée en mètres positifs, et les valeurs de vitesse ou de température gardent une décimale.

Le pas de la grille est aussi calculé dynamiquement. Pour l'axe du temps, le code choisit un pas lisible parmi une liste de valeurs fixes, par exemple 5, 10, 30 ou 60 secondes, puis des valeurs plus grandes si la plongée est longue. Pour l'axe vertical, le pas est choisi selon une logique classique en base 10 avec des valeurs de type 1, 2, 5 ou 10 multipliées par une puissance de 10. Cela évite d'avoir une grille trop dense sur un petit écran, ou au contraire trop peu de repères lorsque le graphe est agrandi.

Le graphe est interactif. Un `DragHandler` permet de déplacer la zone visible, et un `PinchHandler` permet de zoomer ou dézoomer avec deux doigts. Les déplacements sont limités par les bornes réelles des données, afin d'éviter que l'utilisateur puisse sortir complètement du profil de plongée. Cette limitation est faite par la fonction `setClampedView()`, qui recalcule les bornes des axes en les gardant dans les bornes minimales et maximales du graphe.

Un appui simple dans la zone du graphe permet de sélectionner un échantillon. Le point touché à l'écran est converti en coordonnées de données avec la méthode `dataPointCoordinatesAt`. Ensuite, `SampleModel` recherche l'échantillon le plus proche, principalement selon le temps, mais en tenant aussi compte de l'ordonnée affichée. Les informations du point sont renvoyées au QML sous forme de `QVariantMap` : temps, profondeur, température disponible ou non, et vitesse verticale. Ces informations

sont affichées dans le composant *DiveSampleCallout.qml*, sous la forme d'une petite fenêtre placée près du point sélectionné.

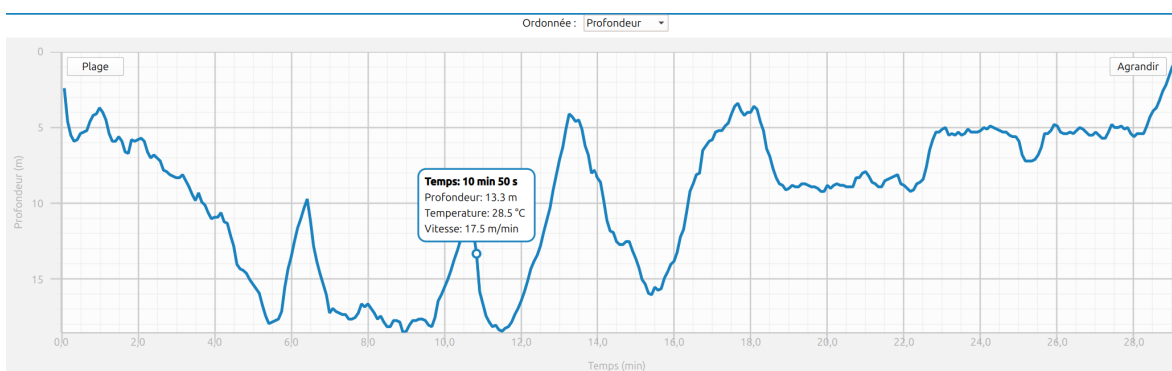


FIGURE 8 – Sélection d'un échantillon et affichage des valeurs associées

Le composant permet aussi de sélectionner une plage de la plongée. Le bouton **Plage** active un mode particulier : le premier appui sélectionne le début de la zone, et le second appui sélectionne la fin. Le modèle C++ calcule alors la durée de la plage, la différence de profondeur entre les deux points et la vitesse moyenne sur cet intervalle. Côté QML, la zone sélectionnée est représentée par une bande semi-transparente entre les deux points, avec deux marqueurs et une bulle d'information spécifique. Cette fonctionnalité permet par exemple d'analyser rapidement une phase de descente ou de remontée.

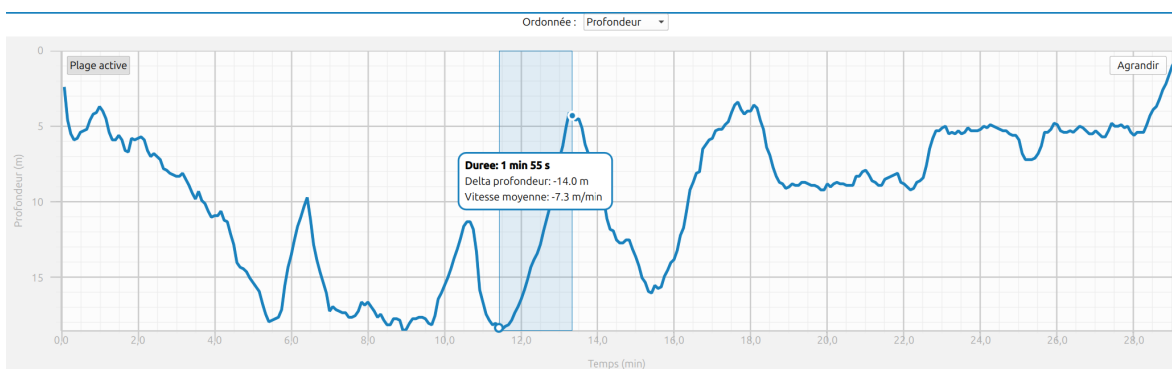


FIGURE 9 – Sélection d'une plage sur le profil de plongée

Les fichiers principaux liés à l'affichage des plongées et des graphiques sont donc :

- *qml/pages/DiveList.qml* : affichage de la liste des plongées disponibles en base.
- *qml/pages/DiveDetails.qml* : page de détail d'une plongée, contenant le graphe et les informations générales.
- *qml/pages/DiveGraphPage.qml* : page dédiée à l'affichage agrandi du graphe.
- *qml/components/DiveGraphView.qml* : composant principal du graphe, qui gère l'affichage, les axes, le zoom, le déplacement et les interactions.
- *qml/components/DiveSampleCallout.qml* : composant affichant les détails d'un point ou d'une plage sélectionnée.

- *src/samplemodel.h* et *src/samplemodel.cpp* : modèle C++ qui charge les échantillons depuis la base, transforme les données en points affichables et calcule les informations utilisées par l'interface.

3.9 Spécificités Android et JNI

L'utilisation des connexions USB sur Android n'a absolument rien à voir avec ce qu'on retrouve sur un ordinateur quelque soit le système d'exploitation. Sur ordinateur on peut exploiter les fichiers d'interface USB/UART (souvent dans */dev/* sous le nom *tty.** sous Linux et MacOS) comme on le veut ce qui rend la tâche très facile. En revanche Android requiert l'autorisation du système pour pouvoir voir et interagir avec ces interfaces. C'est pourquoi il faut pouvoir demander au système Android cette autorisation et traiter le problème ensuite.

Le problème étant largement hors de nos compétences, nous avons réutilisés des fichiers de code issus directement du [GitHub officiel de SubSurface](#), notamment *SubsurfaceMobileActivity.java*, *AndroidSerial.java* et *serial.usb.android.cpp* avec son header. Ces fichiers conservent volontairement leur package d'origine afin de rester compatibles avec les signatures JNI utilisées par le code natif.

Le rôle de cette couche est de détecter les périphériques USB, demander les permissions Android nécessaires, puis fournir à *libdivecomputer* un flux de communication utilisable comme un port série. Le code C++ dialogue alors avec les classes Java via JNI (L'interface Native Java qui permet d'exploiter du code java depuis un autre langage comme C++)

Le recours aux classes Java est une nécessité technique : l'API d'Android pour la gestion de l'hôte USB est exposée au niveau du Framework (en Java), tandis que le noyau du système (un Kernel Linux écrit en C) restreint l'accès direct au matériel pour des raisons de sécurité.

4 Annexes

4.1 Annexe 1 - Script de compilation de *libdivecomputer* pour Android

```
#!/bin/bash

# 1. OS AND TOOLCHAIN DETECTION
OS_TYPE=$(uname -s | tr '[:upper:]' '[:lower:]')
if [ "$OS_TYPE" == "darwin" ]; then
    HOST_TAG="darwin-x86_64"
    CPU_COUNT=$(sysctl -n hw.ncpu)
else
    HOST_TAG="linux-x86_64"
    CPU_COUNT=$(nproc)
fi

# 2. SET PATHS
export NDK_PATH=$HOME/Library/Android/sdk/ndk/27.2.12479018
export TOOLCHAIN=$NDK_PATH/toolchains/llvm/prebuilt/$HOST_TAG

# 3. SET TARGET ARCHITECTURE
export TARGET=aarch64-linux-android
export API=24
export AR=$TOOLCHAIN/bin/llvm-ar
```

```

export AS=$TOOLCHAIN/bin/llvm-as
export CC=$TOOLCHAIN/bin/$TARGET$API-clang
export CXX=$TOOLCHAIN/bin/$TARGET$API-clang++
export LD=$TOOLCHAIN/bin/ld
export RANLIB=$TOOLCHAIN/bin/llvm-ranlib
export STRIP=$TOOLCHAIN/bin/llvm-strip

# 4. CONFIGURE THE BUILD
./configure \
    --host=$TARGET \
    --with-sysroot=$TOOLCHAIN/sysroot \
    --prefix=$(pwd)/dist/android_arm64 \
    --enable-shared \
    --disable-static

# 5. COMPILE AND INSTALL
make clean
make -j$CPU_COUNT
make install

echo "Build Complete pour $OS_TYPE !"

```

Ne pas oublier d'ajuster *NDK_PATH* à la version installée sur votre machine. La version est visible dans le chemin du NDK dans les préférences de *Qt Creator*.

De même, pour viser une version différente de Android, modifier les paramètres sous "SET ARCHITECTURE"